

Mobile Development Demonstration + 3rd Party Integ

LAB 10/W12

3ND YEAR - ENSIA

IMPORTANT

- If you are found to use any LLM (including copilote) inside the lab and you are warned three times, expect **zero mark for your CC**.

OBJECTIVES

- Students demonstrate their mobile application to get feedback from fellow students.
- Students need to learn how to integrate simple Cloud-based services for storage.

DEMONSTRATION

Task / Exercise C1 [90 minutes demonstration] :

Every team will be given strictly 15 minutes to showcase their app to other teams presenting the following :

- Clear and Brief : Mission statement from their App.
- Who are their target audience
- What value being given to their audience
- Real demonstration running on mobile phones/emulators shown on a projector to other students showing :
 - The primary use cases being implemented
 - Use of a local database
 - Use of Cubit/BloC
 - Localisation.
- NO NEED FOR THE BUSINESS OR MONETISATION PROCESS.

As a Lab Supervisor, we will ensure that :

- Students are progressing and collaborating smoothly on their mobile dev project
- Students are learning the different concepts explained in previous sessions
- There will be questions to assess students' knowledge.

INDIVIDUAL EXERCISES

Task / Exercise C2 (Run & Understand) [10 minutes + 5 minutes discussion]:

Download and run the following :

https://drive.google.com/file/d/1k2BmWXJVD19ZXfA6-1LEFBr8_JmZ3yuN/view?usp=drive_link

Explore the Use of SupaBase for the Todo App.

Task / Exercise C3 (Use of External Services) [75 minutes]:

The MedBox App as the case study, its profile section is using a local database via the Repository Pattern.

Inside the main file, it is registered with the service locator :

```
final sl = GetIt.instance;

sl.registerLazySingleton<ProfilesRepoAbstract>(() =>
ProfilesRepoLocalDB());

sl.registerLazySingleton<MedicalRecordsRepo>(() =>
MedicalRecordsRepo());
```

Whilst the concrete implementation of the database version is shown as below:

```
class ProfilesRepoLocalDB extends
ProfilesRepoAbstract {
  final DB_PROF_REC = DBProfilesTable();
  Future<List<Profile>> getData() async {
    List<Map<String, dynamic>> data = await
DB_PROF_REC.getRecords();
    return Profile.fromJsonList(data);
  }
  ...
}
```

A new model is created as a class to store the profile data instead of using Map whilst using the to assist with the auto generation of utility methods...

```
import 'package:json_annotation/json_annotation.dart';
part 'profile.g.dart';
@JsonSerializable()
class Profile {
  final int? id;
  final String name;
  final String dob;
  Profile({this.id, required this.name, required this.dob});
  ...
}
```

Download the following Project:

https://drive.google.com/file/d/1oUPBxZ9PSbTszcblcRQ06B9u5Bn9qmu7/view?usp=drive_link

Your task, **extend** the code to load or save the **profile data** from/to:

- First, Supabase (You have to create your own project)
- and later Firestore

No need to create a backend, relax the security rules checks.

Extend : means create a separate repo implementation for the corresponding technology.

Note for Firebase ? you may change the Model ? You need to regenerate the ...

